

Java Streams / Collectors / Comparator Cheat Sheet

The core question before you write any code

What shape is my data now, and what shape do I want next?"

- Processing **objects** → `Stream<T>`
- Doing **math** on numbers → `IntStream` / `LongStream` / `DoubleStream`
- Building a **container** (List/Map/String/grouped data) → `Collector` → feeds `.collect(...)`
- Defining an **ordering rule** → `Comparator` → feeds `.sorted(...)`

1. Stream<T> — core methods

Method	What it does	Returns
<code>filter(Predicate<T>)</code>	keep elements matching condition	<code>Stream<T></code>
<code>map(Function<T, R>)</code>	transform each element	<code>Stream<R></code>
<code>flatMap(Function<T, Stream<R>>)</code>	flatten nested structure (e.g. <code>List<List<T>></code> → <code>List<T></code>)	<code>Stream<R></code>
<code>sorted()</code> / <code>sorted(Comparator)</code>	order elements	<code>Stream<T></code>
<code>distinct()</code>	remove duplicates (uses <code>.equals()</code>)	<code>Stream<T></code>
<code>limit(n)</code> / <code>skip(n)</code>	truncate / drop first n	<code>Stream<T></code>
<code>peek(Consumer)</code>	debug/side-effect mid-pipeline (avoid for real logic)	<code>Stream<T></code>
<code>forEach(Consumer)</code>	terminal — do something with each element	<code>void</code>
<code>reduce(...)</code>	terminal — combine all elements into one value	<code>Optional<T></code> / <code>T</code>
<code>collect(Collector)</code>	terminal — build a container	varies

Method	What it does	Returns
<code>anyMatch()/allMatch()/noneMatch(Predicate)</code>	terminal — boolean check	<code>boolean</code>
<code>findFirst()/findAny()</code>	terminal — get one element	<code>Optional<T></code>
<code>count()</code>	terminal — how many elements	<code>long</code>
<code>toList()</code> (Java 16+)	terminal shortcut for <code>collect(toList())</code>	<code>List<T></code>

Bridges to primitive streams:

```

java

stream.mapToInt(T::someIntField)    // Stream<T> -> IntStream
intStream.boxed()                   // IntStream -> Stream<Integer>

```

2. IntStream / LongStream / DoubleStream — only for math

Use these the moment you're summing, averaging, or finding min/max of raw numbers.

Method	What it does	Returns
<code>sum()</code>	total	<code>int</code> / <code>long</code> / <code>double</code>
<code>average()</code>	mean	<code>OptionalDouble</code>
<code>max()/min()</code>	extremes	<code>OptionalInt</code> /etc.
<code>summaryStatistics()</code>	count, sum, min, max, avg all at once	<code>IntSummaryStatistics</code>
<code>range(a, b)/rangeClosed(a, b)</code>	generate a sequence of ints	<code>IntStream</code>
<code>boxed()</code>	convert back to <code>Stream<Integer></code>	<code>Stream<Integer></code>

```

java

```

```
// Object list -> math -> IntStream
int totalLen = words.stream().mapToInt(String::length).sum();

// Generate a range
IntStream.rangeClosed(1, 5).forEach(System.out::println); // 1 2 3 4 5
```

Rule: if you're not doing arithmetic, stay in `Stream<T>` — don't convert to primitive streams just because the values happen to be numbers (e.g. sorting `Integer`s doesn't need `IntStream`).

3. Collectors — grouped by "what shape do I want to end with?"

Import once: `import static java.util.stream.Collectors.*;`

I want...	Collector
A List	<code>toList()</code>
A Set	<code>toSet()</code>
A specific Map	<code>toMap(keyFn, valueFn)</code>
A joined String	<code>joining()</code> , <code>joining(", ")</code> , <code>joining(", ", "[", "]"")</code>
Just a count	<code>counting()</code>
Sum / average of a field	<code>summingInt/Long/Double(fn)</code> , <code>averagingInt/Long/Double(fn)</code>
Min / max by a field	<code>minBy(Comparator)</code> , <code>maxBy(Comparator)</code>
Groups: key → List	<code>groupingBy(keyFn)</code>
Groups: key → (custom downstream)	<code>groupingBy(keyFn, downstreamCollector)</code>
Groups: key → transformed List	<code>groupingBy(keyFn, mapping(valueFn, toList()))</code>
Split into two buckets (true/false)	<code>partitioningBy(predicate)</code>
Combine multiple stats in one pass	<code>teeing(collectorA, collectorB, combiner)</code> (Java 12+)

The pattern that unlocks most "hard" problems: nested collectors

```
java

// group by dept, then just the names
groupingBy(Person::getDept, mapping(Person::getName, toList()))

// group by dept, then count
groupingBy(Person::getDept, counting())

// group by dept, then average salary
groupingBy(Person::getDept, averagingDouble(Person::getSalary))

// group by dept, then the highest-paid person in each
groupingBy(Person::getDept, maxBy(Comparator.comparing(Person::getSalary)))
```

Read `groupingBy(X, Y)` as: "group by X, then do Y to each group."

toMap — always a plain Function, never a Collector

```
java

toMap(Person::getName, Person::getSalary)           // simple key->value
toMap(Person::getDept, Person::getName, (a, b) -> a) // 3rd arg = merge fun
```

If you feel like you need a `Collector` as the 2nd argument of `toMap`, you actually want `groupingBy(keyFn, downstreamCollector)` instead.

4. Comparator — build it like a sentence

```
java

Comparator.comparing(Person::getAge)           // ascending by age
    .thenComparing(Person::getName)           // tie-break: ascend
    .reversed();                               // flips the WHOLE
```

Method	What it does
<code>Comparator.comparing(fn)</code>	sort by a field, ascending
<code>Comparator.comparing(fn, otherComparator)</code>	sort by a field using a custom rule (e.g. reverse just this field)
<code>.reversed()</code>	flips everything before it in the chain
<code>.thenComparing(fn)</code>	tie-breaker field
<code>Comparator.reverseOrder()</code>	ready-made descending natural order (pass as 2nd arg to <code>comparing</code>)
<code>Comparator.naturalOrder()</code>	ready-made ascending natural order
<code>Collections.reverseOrder()</code>	same idea, for use with <code>Collections.sort</code>

Mixing directions per-field (common gotcha)

```

java

// age DESC, name ASC – different directions on different fields
Comparator.comparing(Person::getAge, Comparator.reverseOrder())
    .thenComparing(Person::getName)

```

`.reversed()` at the end reverses everything — only use it when you want **every** field flipped.

Comparator feeds `.sorted(...)`. It never goes into `.collect(...)`.

5. Decision cheat-table (use this when stuck)

You're thinking...	You actually need
"sum / average / max of numbers"	<code>mapToInt/Double(...)</code> then <code>.sum()/ .average()/ .max()</code>
"turn stream into a List/Set/Map"	a <code>Collector</code> → <code>.collect(...)</code>
"group things by a key"	<code>groupingBy(...)</code>
"group AND aggregate within each group"	<code>groupingBy(key, downstreamCollector)</code>

You're thinking...	You actually need
"order elements"	a <code>Comparator</code> → <code>.sorted(...)</code>
"order by 2+ fields"	<code>Comparator.comparing(...).thenComparing(...)</code>
"keep only some elements"	<code>.filter(...)</code>
"transform each element"	<code>.map(...)</code>
"flatten nested lists"	<code>.flatMap(...)</code>
"build a Map from a Collector's 2nd arg" (feels wrong)	you want <code>groupingBy</code> , not <code>toMap</code>

6. The habit that prevents the confusion

Before writing code, write one English sentence:

| "I have , I want to **group by** / **sort by** / **sum** / **filter** , then .

Translate word-for-word:

- "group by X" → `groupingBy(X)`
- "then Y" → downstream collector or `.thenComparing`
- "sum/average of Z" → primitive stream math, or a summing/averaging Collector if inside a `groupingBy`